

Universidade do Minho
Licenciatura em Engenharia Informática

Unidade Curricular de
Inteligencia Artificial
3 de Janeiro 2025

Trabalho Prático

Resolução de Problemas- Algoritmos de procura

Sistema de Distribuição de Recursos em Cenários de Catástrofe

- André Costa, A80824;
- Maria Leonor Carvalho da Cunha, A103997;
- João Pedro Ferreira e Ferreira, A104539;
- Gonçalo Rocha Sousa Freitas, A104350;



Índice

1. Introdução **3**
2. Descrição do Problema **3**
3. Formulação do Problema **4**
3.1 Estado Inicial 4
3.2. Teste Objetivo 4
3.3. Operadores (Ações Disponíveis) 4
3.4. Função de Custo 5
4. Tarefas Realizadas **5**
4.1 Construção dos Grafos 5
4.2 Estratégia adotada - Algoritmos de Procura 8
4.3 Condições meteorológicas dinâmicas 10
4.4 Menu 10
4.5 Entrega de suprimentos 12
6. Resultados e Discussão **12**
6.1. Resultados Obtidos: 12
6.2. Limitações: 13
6.3. Discussão: 13
7. Conclusão **13**
8. Referências **13**

1. Introdução

Este relatório apresenta o desenvolvimento e os resultados obtidos no projeto intitulado *Sistema de Distribuição de Recursos em Cenários de Catástrofe.*

A distribuição de alimentos e suprimentos durante uma catástrofe natural é um desafio crítico que exige planos eficientes e decisões rápidas. Este projeto tem como objetivo aplicar algoritmos de procura para desenvolver soluções otimizadas que garantam a utilização eficiente de recursos, priorizando as áreas mais necessitadas dentro de um intervalo de tempo restrito.

Assim, neste trabalho, desenvolvemos um programa em python capaz de otimizar a distribuição de recursos para áreas afetadas, considerando diversos fatores como gravidade, população e condições dinâmicas.

2. Descrição do Problema

Em cenários de catástrofes naturais, a distribuição eficaz de recursos é essencial para mitigar os danos e salvar vidas. Este problema apresenta os seguintes desafios:

- Identificar as zonas afetadas, avaliando sua população e a gravidade da situação.
- Modelar as rotas de transporte considerando distância, tempo e condições de acessibilidade.
- Selecionar os veículos mais adequados para transporte, levando em conta suas capacidades e limitações.
- Adaptar as decisões dinamicamente, conforme mudanças nas condições climáticas e bloqueios de rotas.

3. Formulação do Problema

O problema foi modelado como um grafo G = (N, E), onde:

- N é o conjunto de nós que representam as zonas afetadas.
- E é o conjunto de arestas que representam as rotas entre as zonas, com atributos como distância, tempo e acessibilidade.

3.1 Estado Inicial

O estado inicial é composto por os veículos estarem todos na posição inicial, na zona1, abastecidos e prontos para a qualquer momento serem utilizados. Para além disso, os suprimentos, tais como, alimentos, água e medicamentos, encontram-se armazenados de forma segura mas também acessível, com vista a minimizar o tempo quando forem necessários serem transportados. Existem informações iniciais sobre as zonas afetadas, a sua localização e estado e nenhuma zona além da 1 foi visitada.

3.2. Teste Objetivo

Maximizar o atendimento às zonas afetadas com maior gravidade e densidade populacional, minimizando o desperdício de recursos e cumprindo as restrições de tempo. Isto passa por, verificar que todas as zonas recebem os suprimentos; realizar as entregas dentro da janela de tempo crítica de cada zona; minimizar o total de tempo de entrega, sendo eficientes; otimizar o uso de combustível. Além disso, foram consideradas as seguintes restrições: capacidade de carga e autonomia dos veículos e acessibilidade das rotas para diferentes tipos de veículos.

3.3. Operadores (Ações Disponíveis)

- Seleção de Veículo: Escolha entre drone, camião, helicóptero, barco e mota. Para além disso, cada veículo tem características específicas (capacidade, velocidade, autonomia). Existem restrições de acessibilidade baseadas no terreno.
- Movimentação entre Zonas: Deslocamento entre zonas conectadas. Sujeito a restrições de acessibilidade do veículo e o custo é baseado em distância e tempo de viagem.
- Seleção de Rota: Escolha entre diferentes algoritmos (A*, Dijkstra, DFS, BFS,

Greedy). Há consideração de fatores como: População da zona; Gravidade da situação; Tempo crítico de resposta; Autonomia restante do veículo.

- Adaptação a Condições: Resposta a alterações climáticas (aumenta tempo de viagem). Gestão de bloqueios de rotas e atualização dinâmica de caminhos disponíveis.

3.4. Função de Custo

Minimizando o custo total e cumprindo todos os objetivos, o custo da solução pode ser formulado considerando:

1. Tempo total de viagem: Tempo base de cada rota é ajustado pela velocidade do veículo e pode ser influenciado por condições climáticas (aumento de 50% em condições adversas).
2. Recursos consumidos: Autonomia do veículo utilizada; Capacidade de carga necessária.
3. Prioridades: População afetada; Nível de gravidade da zona (1-10); Tempo crítico para atendimento.

A função de custo está implementada através dos pesos calculados em *encontrar_melhores_rotas():*

custo_caminho = dados['distancia'] / veiculo['velocidade']
score_ajustado = scores_zonas[zona] / custo_caminho

Onde *scores_zonas* é calculado como:
*scores_zonas = (grafo.nodes[zona]['populacao'] * grafo.nodes[zona]['gravidade']) / 100*

---

4. Tarefas Realizadas

4.1 Construção dos Grafos

De forma a podermos utilizar os algoritmos de procura, representamos as zonas, os caminhos e os veículos em grafo.

1. Zonas

As zonas (no nosso projeto existem 100 zonas) são representadas como

vértices do grafo e possuem os seguintes atributos:

- Nome: Identificação única da zona (ex.: zona1, zona2);
- População: Quantidade de pessoas na zona;
- Gravidade: Nível de necessidade da zona (1 a 10);
- Tempo Crítico: Tempo limite para atendimento antes que a situação se torne inviável;
- Posição: Coordenadas para visualização no grafo.

Exemplo de dados:

nome	populacao	gravidade	tempo_critico	pos_x	pos_y
zona1	3222	3	41	55.16922898202803	93.09790656347779
zona2	4601	8	27	18.160326162921546	97.2967214742724
zona3	1495	5	54	80.71035380501868	40.81745328607844
zona4	4768	7	65	4.217205715028749	99.77960941090109

Imagem 1 - Exemplos de dados das Zonas

2. Caminhos

Os caminhos unem as zonas e são representados como arestas do grafo com os seguintes atributos:

- Distância: Distância entre as zonas;
- Tempo: Tempo de viagem estimado;
- Acessibilidade: Tipos de veículos que podem aceder ao caminho.

Exemplo de dados:

zona1	zona2	distancia	tempo	acessibilidade
zona1	zona36	28.66	6.89	8
zona1	zona57	10.9	8.62	6
zona1	zona97	65.86	1.07	10
zona2	zona80	95.42	2.62	9
zona2	zona8	10.71	5.5	4

Imagem 2 - Exemplos de dados dos Caminhos

3. Veículos

Os veículos disponíveis têm as seguintes características:

- Tipo: Identificação do veículo (ex.: drone, camião);
- Capacidade: Quantidade de suprimentos possui num instante;
- Capacidade Total: Quantidade de suprimentos que pode transportar;
- Velocidade: Velocidade média do veículo;
- Autonomia: Distância máxima que pode percorrer antes de reabastecer;
- Acessibilidade: Tipos de terrenos ou caminhos que o veículo pode aceder.

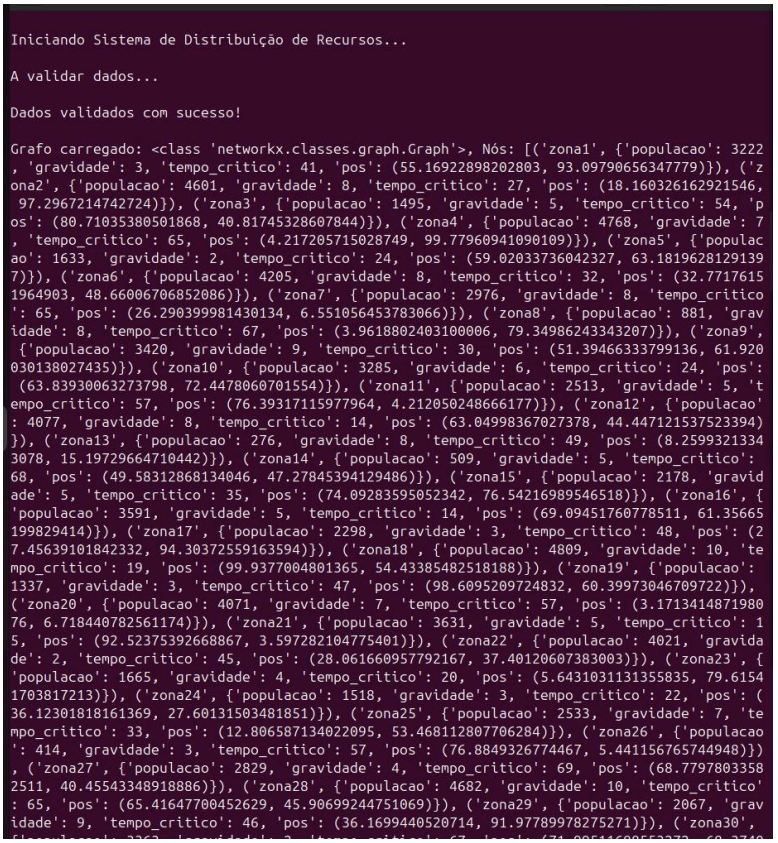
Exemplo de dados:

tipo	capacidade	velocidade	autonomia	acessibilidade
drone	10	50	20	drone
camiao	2000	60	500	camiao

Imagem 3 - Exemplo de dados de veículos

Todos estes dados são validados através do ficheiro *validacao.py* para depois serem usados aquando da execução do trabalho. A função *validar_dados_csv()* verifica se todos os dados contidos nos *DataFrames (zonas_df, caminhos_df e veículos_df*) estão válidos, que não tem falhas nenhumas e retorna uma lista de erros encontrados; a função *testar_conetividade_grafo()* verifica se o grafo é conectado, se todas zonas do grafo são acessíveis entre si. Para além disso, a função *imprimir_veiculos()* exibe a lista de veículos carregados com sucesso. Estas funções ajudam a garantir que os dados sejam consistentes e que a rede modelada esteja corretamente conectada antes de ser utilizada.

A partir do ficheiro *grafo.py* o grafo é carregado e desenhado utilizando as funções: *carregar_grafo()* - esta função utiliza as informações dos ficheiros csv zonas e caminhos para construir um grafo utilizando a biblioteca *networkx,* resumidamente, modela um sistema geográfico com acessibilidade e características específicas de cada zona e caminho - e *mostrar_grafo()* - extrai as posições dos nós e desenha o grafo, adicionando também as distâncias.



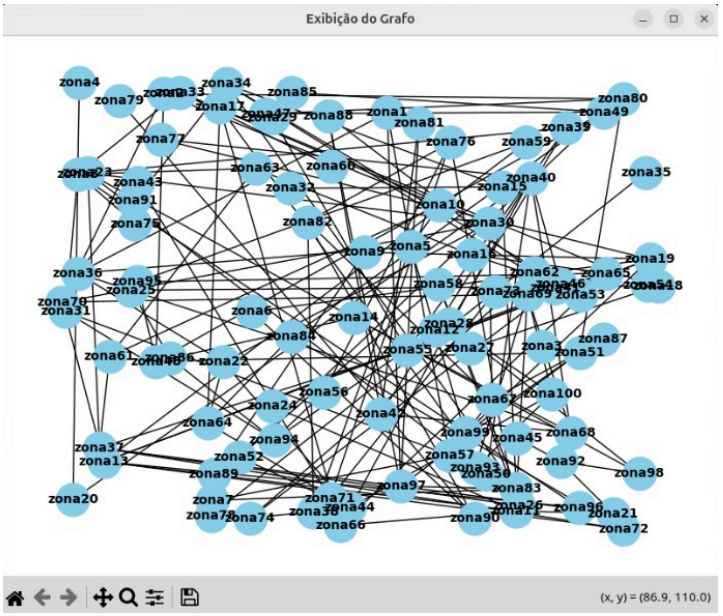


Imagem 4 e 5 - Print de uma parte do terminal que mostra o grafo carregado e os dados; Grafo gerado

4.2 Estratégia adotada - Algoritmos de Procura

Estratégias de procura não informadas usam apenas informações disponíveis na definição do problema. Nas estratégias de procura informadas dá-se ao algoritmo “dicas” sobre a adequação de diferentes estados.

Neste projeto foi-nos proposto o desenvolvimento de estratégias de procura, informada ou não informada. Nós decidimos implementar cinco estratégias de procura, duas delas não informadas e três informadas.

Primeiramente implementamos o algoritmo Dijkstra (procura informada), depois o algoritmo A* (procura informada). De seguida decidimos implementar o algoritmo BFS (*Breadth-First-Search*), DFS (*Deep-First-Search*) (os dois de procura não informada) e *Greedy/Gulosa* (procura informada).

A procura BFS é normalmente melhor para problemas mais pequenos, no entanto, é esperado que o tempo de execução desta seja maior pois analisa todos os nós de menor profundidade. Na procura DFS é possível obter um tempo de execução menor pois não necessita de muita memória para tal, porém também é esperado uma solução menos ótima.

Os algoritmos informados que implementamos, dois deles, utilizam heurística e esta é calculada a partir da distância euclidiana entre duas zonas [((x1 - x2) ** 2 + (y1 - y2) ** 2) ** 0.5] (*heuristica()*).

A estratégia adotada no algoritmo de Dijkstra implementado foi adaptada para atender às especificidades do problema, considerando a acessibilidade das rotas por tipo de veículo e o tempo de deslocamento como critério para cálculo dos pesos das arestas. O grafo foi estruturado de forma que cada aresta contém informações detalhadas sobre a acessibilidade para diferentes tipos de veículos e o tempo base necessário para percorrê-la. Durante a execução, o algoritmo verifica, para cada aresta adjacente, se o veículo em questão possuía acesso permitido pela configuração da rota. O funcionamento geral do algoritmo foi eficaz em identificar o melhor caminho em termos de tempo, respeitando as restrições impostas pela acessibilidade e pelo tipo de veículo. No entanto, a eficiência pode variar dependendo do tamanho do grafo e da densidade de suas conexões, especialmente considerando os cálculos adicionais relacionados à velocidade relativa do veículo. Em testes realizados, o algoritmo foi capaz de fornecer trajetos otimizados e consistentes, demonstrando que a adaptação do peso por tempo e velocidade do veículo é uma estratégia funcional para problemas de roteamento em grafos com restrições.

No caso da procura A*, os resultados apresentados são bons se as heurísticas forem bem definidas, daí termos definido a heurística como referido acima, tem em conta a distância percorrida até um nodo e a heurística do mesmo para escolher a rota a seguir, encontra o caminho mais curto considerando as restrições, devolve bons resultados. A Gulosa, no entanto, apenas tem em conta a heurística dos nodos. Esta não tem em conta o custo das ligações e nestes casos os custos entre os nodos são diferentes e uns maiores que outros não gera caminhos muito satisfatórios.

A partir destas cinco estratégias de procura diferentes é possível vermos as diferenças entres eles e comparar qual o melhor para o efeito pretendido.

4.3 Condições meteorológicas dinâmicas

No ficheiro *simulacao.py* implementamos a função *simular_cenario()* que simula alterações climáticas e o bloqueio de rotas. Assim, em alguns caminhos, a chance de mau tempo é de 30% aumentando o tempo de viagem de uma zona para a outra 150%. De forma também aleatória com uma probabilidade de 20% algumas rotas ficam bloqueadas e por isso inutilizadas pelos veículos. Todas as rotas que sofrem estas alterações são apresentadas no terminal como podemos ver na imagem 6, onde podemos visualizar o tempo de viagem atualizado, no caso das mudanças meteorológicas, ou com o simples aviso que uma rota está bloqueada.

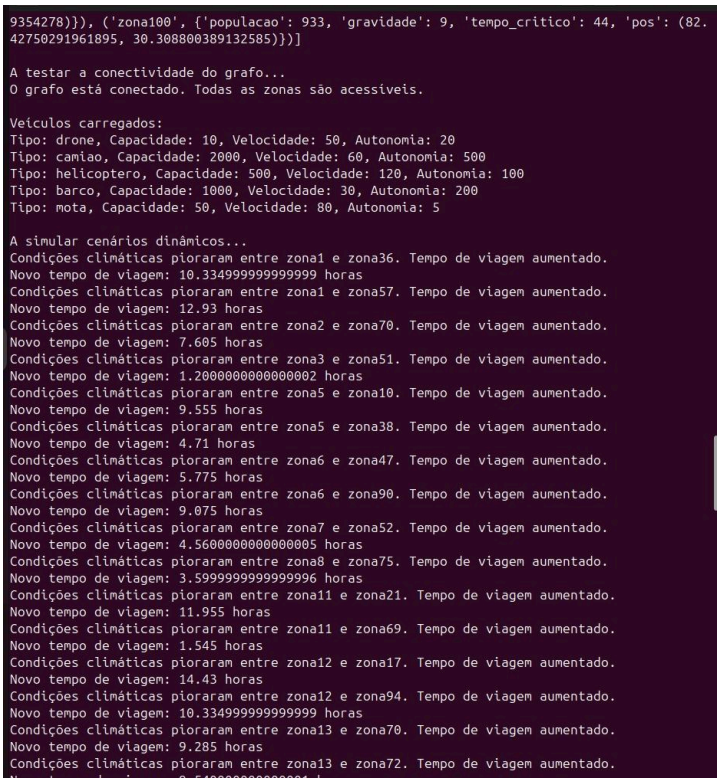


Imagem 6 - Print de uma parte do terminal onde é simulado um cenário dinâmico

4.4 Menu

Decidimos implementar um menu que permite ao utilizador da aplicação escolher de forma mais interativa o que pretende fazer.

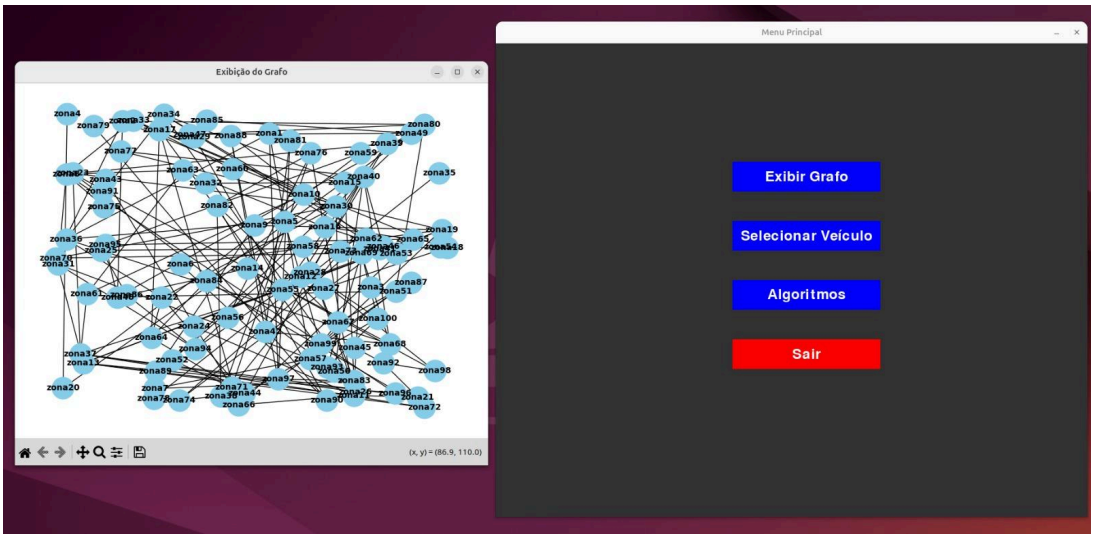


Imagem 7- Menu geral

Este menu possui as seguintes funcionalidades:

1. Exibir Grafo: cria uma janela com o grafo das zonas criadas. (ver imagem 5).
2. Selecionar veículo: mostra as possibilidades dos veículos que podemos escolher para calcular o melhor trajeto, sabendo que cada veículo possui as suas próprias características. Deve ser selecionado primeiro um veículo e só depois o algoritmo, se tal não acontecer é exibida uma mensagem de erro como podemos ver na imagem 9.

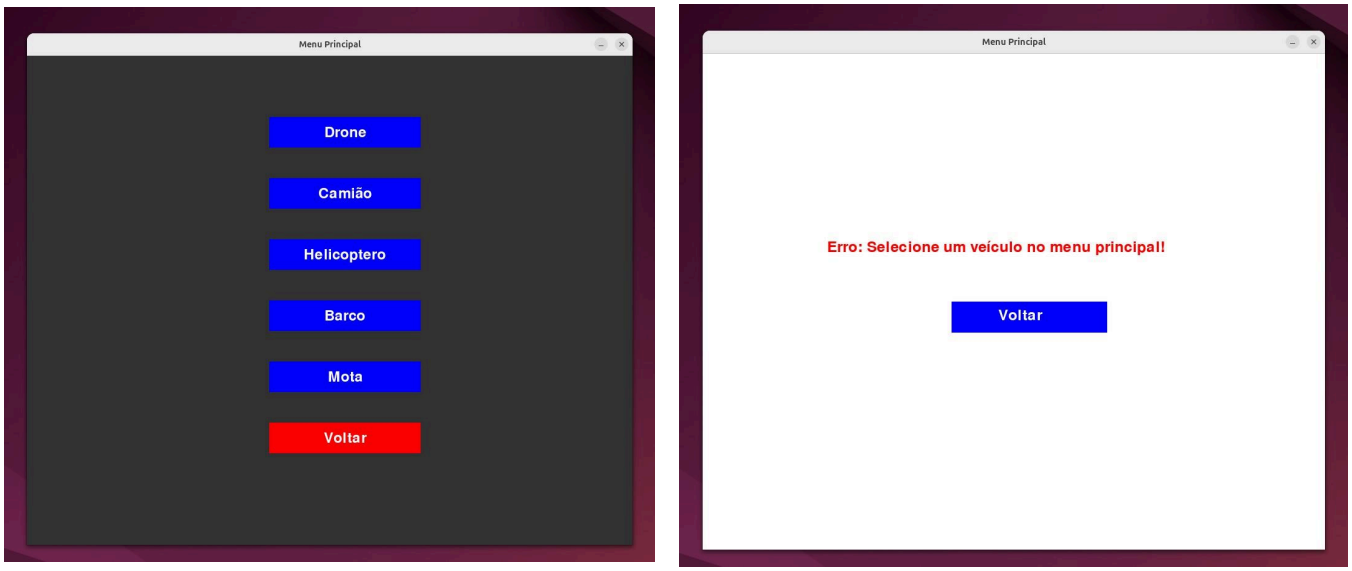


Imagem 8 e 9 - Menu dos veículos e mensagem de erro, respetivamente.

3. Selecionar o Algoritmo que quer utilizar. Após selecionado o programa apresenta os resultados no terminal e em janelas de visualização com os grafos e as rotas representadas.

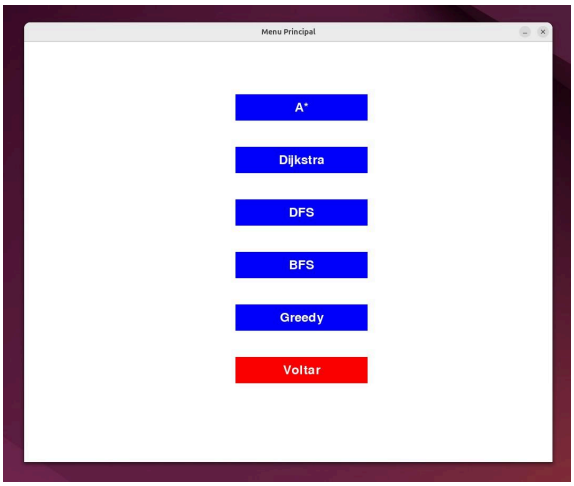


Imagem 10 - Menu de seleção do algoritmo

4. Sair e fechar o programa.

4.5 Entrega de suprimentos

Como foi explicado anteriormente cada veículo possui uma capacidade e uma capacidade total, cada zona possui uma capacidade mínima, ou seja, caso um veículo for a essa zona deve deixar lá a quantidade mínima de suprimentos para essa zona. Decidimos que inicialmente todos os veículos possuíam capacidade igual à capacidade total dos mesmos, mas sempre que um veículo percorre uma zona ele perdia “capacidade” por deixar suprimentos nessa zona, assim sempre que o sistema queria decidir se um veículo deveria ir a uma zona tomava consideração a sua capacidade. Além de isso, se acontecer o acaso do veículo passar por uma base ele reabastece a sua capacidade voltando à sua capacidade e autonomia máxima.

---

6. Resultados e Discussão

6.1. Resultados Obtidos:

- *A:** Melhor balanço entre tempo de execução e eficiência.

- Dijkstra: Robusto para encontrar o caminho de menor custo, mas com maior complexidade computacional.
- Greedy: Rápido, mas incapaz de lidar com cenários dinâmicos eficientemente.
- DFS e BFS: Adequados para cenários simples, mas ineficazes em problemas maiores.

6.2. Limitações:

- Dependência de dados precisos sobre condições meteorológicas e acessibilidade.
- Complexidade computacional em cenários com grande número de zonas.

6.3. Discussão:

Os resultados mostram que algoritmos informados, como A*, são mais adequados para problemas dinâmicos com restrições e prioridades como o que nos foi proposto.

---

7. Conclusão

Este trabalho permitiu-nos consolidar diversos conhecimentos obtidos durante as aulas, como a resolução de problemas implementando algoritmos de procura e relativamente a linguagem python.

Achamos que o trabalho foi desenvolvido de forma igual e num espírito de entreajuda onde todos contribuímos para o trabalho e procuramos tornar cada vez melhor o trabalho. Há coisas que podiam ser melhoradas, tanto no processo de desenvolvimento do trabalho, como na gestão de tempo.

Tivemos algumas dificuldades em pôr o reabastecimento dos veículos a funcionar, o que acabamos por não incluir no trabalho infelizmente. No geral acho que conseguimos cumprir tudo o resto com alguns obstáculos mas foram superados.

---

8. Referências

- Russell, S., & Norvig, P. (2009). *Artificial Intelligence - A Modern Approach*. 3rd Edition.
- Costa, E., & Simões, A. (2008). *Inteligência Artificial - Fundamentos e Aplicações*. FCA.
- A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts*, Wiley.
- NetworkX Documentation: https://networkx.org
- Python Official Website: https://www.python.org/